

ETL-SQL User Manual: Thinking in Pipelines

Welcome to ETL-SQL. This guide is designed to help you transition from thinking in "Single Database SQL" to "Multi-Context Data Flow." Work through each section in order — each one builds on the last.

[!TIP] **Stuck on something specific?** Use the table of contents below to jump directly to the section you need. For a searchable list of errors and gotchas, see the [FAQ](#). For connector-specific syntax, see [Data_Connectors.md](#).

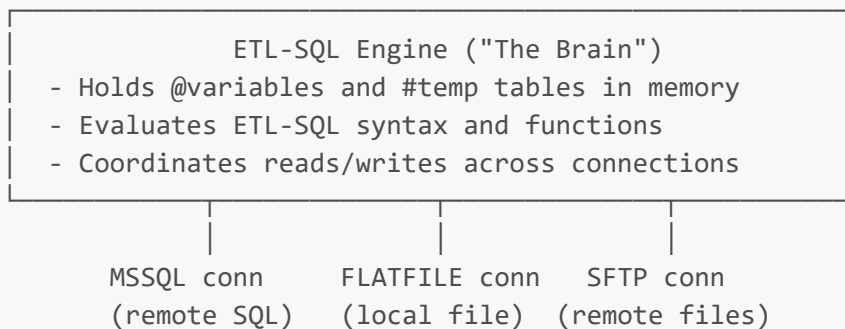
Contents

1. [The Pipeline Mental Model](#)
2. [Your First Connection](#)
3. [Variables & State Management](#)
4. [The #Temp Table Workspace](#)
5. [Core SELECT Patterns](#)
6. [Control Flow](#)
7. [Error Handling & Transactions](#)
8. [Data Movement Patterns](#)
9. [File Operations](#)
10. [Modular Scripts & Jobs](#)
11. [Metadata, Lineage & Tags](#)
12. [Debugging & Diagnostics](#)
13. [Zero-Trust Security](#)

- [Next Steps](#)

1. The Pipeline Mental Model

The most important concept to master is **Context Awareness**. In standard SQL, your query runs against a single engine. In ETL-SQL, you are the **Conductor** of an orchestra of engines.



1.1 Engine Context vs. Remote Context

Context	What runs here	Key examples
Engine	ETL-SQL syntax, @variables, #temp tables, functions, FOREACH, IF, MERGE	SELECT ... INTO #stage FROM conn.Table
Remote	Native SQL of the target engine — passed verbatim	EXECUTE mssql_conn BEGIN ... END

[!IMPORTANT] **The Golden Rule:** Data always flows *through* the engine. If you want to move data from Postgres to a CSV file, you stage it in a #temp table first. This is where validation, masking, regex, and lineage tagging happen. The remote engines only ever receive simple, native SQL they can execute directly.

1.2 Why This Matters

- ETL-SQL functions like REGEXP_LIKE, HASHBYTES, and FORMAT() only work in **engine context** (#temp tables, engine-side SELECT).
- SQL dialect keywords (TOP, GETDATE(), NOW()) are validated against the target connection. Writing SELECT TOP 10 against a Postgres connection causes a lint error — use LIMIT 10 instead.
- When you use EXECUTE conn BEGIN ... END, that block is sent to the remote engine **unchanged** — write pure native SQL for the target inside those blocks.

2. Your First Connection

Every data source is represented by a named **connection**. Create one before querying it.

```
-- Create a connection to SQL Server using Windows auth
CREATE CONNECTION prod ON MSSQL()
    WITH(SERVER='sql01', DATABASE='SalesDB', TRUSTED_CONNECTION=TRUE);

-- Query it like any table
SELECT TOP 100 * FROM prod.dbo.Customers WHERE Status = 'Active';
```

2.1 Connection Syntax Options

```
-- Structured (recommended – readable and diffable)
CREATE CONNECTION mydb ON MSSQL()
    WITH(SERVER='sql01', DATABASE='HR', USER='etl', PASSWORD='s3cr3t');

-- Traditional string (useful for encrypted or DSN strings)
CREATE CONNECTION mydb ON
MSSQL('Server=sql01;Database=HR;User=etl;Password=s3cr3t;');
```

2.2 Encrypting Credentials (ENC:)

Never commit plaintext passwords. Use the master password to encrypt connection strings:

```
USE PASSWORD = 'myMasterSecret';

-- The engine auto-decrypts ENC: strings at connection time
CREATE CONNECTION secure_db ON MSSQL('ENC:U2FsdGVkX1+abc...');
```

[!TIP] The IDE and CLI can automatically encrypt all plaintext connection strings in a script when you save with a master password set. Use `HELP CONNECTION <type>` in the TUI (e.g. `HELP CONNECTION MSSQL`) to see every available `WITH()` option and its default for any connector type.

2.3 Environment Switching

Use `CREATE SETS` to define named groups of variables for different environments or setups:

```
CREATE SETS !DEV
BEGIN
    @server = 'dev-db.local',
    @db      = 'DevWarehouse'
END

CREATE SETS !PROD
BEGIN
    @server = 'prod-db.local',
    @db      = 'ProdWarehouse';
    SET WITH_PROMPT ON; -- requires confirmation in interactive mode
END

USE SETS !DEV;
CREATE CONNECTION dw ON MSSQL() WITH(SERVER=@server, DATABASE=@db,
TRUSTED_CONNECTION=TRUE);
```

3. Variables & State Management

Variables are the engine's memory. Prefix all variable names with `@`.

```
-- Declare with an explicit type (or omit for inferred ANY type)
DECLARE @BatchDate DATE = '2026-04-01';
DECLARE @Threshold DECIMAL = 5000.00;
DECLARE @Label STRING = 'Q2-Load';
DECLARE @ids LIST = (1, 2, 3, 4);

-- Set a new value
SET @Threshold = @Threshold * 1.1;

-- Use in a query
SELECT * FROM prod.Sales WHERE SaleDate >= @BatchDate AND Amount > @Threshold;
```

3.1 INPUT and OUTPUT Parameters

Variables can be marked **INPUT** (overridable from CLI or parent script) or **OUTPUT** (returns a value to the caller):

```
-- In a sub-script
DECLARE @Env      STRING INPUT  = 'DEV';    -- CLI: --var @Env=PROD
DECLARE @RowsOut INT    OUTPUT = 0;

SET @RowsOut = (SELECT COUNT(*) FROM #staging);

-- In the parent script
DECLARE @Loaded INT;
RUN SCRIPT 'ingest.etlsql' WITH (@Env = 'PROD', @Loaded = @Loaded);
PRINT 'Loaded rows: ' + @Loaded;
```

3.2 Environment Sets — Switching DEV / QA / PROD

Instead of changing connection strings throughout your script, define **named environment sets** once and activate them with a single command:

```
-- Define environments once (usually stored in a shared setup script)
CREATE SETS !DEV
BEGIN
    @server    = 'dev-db.internal',
    @database  = 'DevWarehouse'
END

CREATE SETS !PROD
BEGIN
    @server    = 'prod-db.internal',
    @database  = 'ProdWarehouse';
    SET WITH_PROMPT ON;    -- requires confirmation before activating
END

-- Activate the environment for this run
USE SETS !DEV;

-- Now use the variables
CREATE CONNECTION dw ON MSSQL() WITH(SERVER=@server, DATABASE=@database,
TRUSTED_CONNECTION=TRUE);

-- Switch to PROD (prompts for confirmation in interactive mode)
USE SETS !PROD;

-- Remove a set that is no longer needed
DROP SETS IF EXISTS !STAGING;
```

[!TIP] **CREATE SETS** blocks are typically placed in a shared `_environments.etlsql` and loaded at the top of each orchestrator script via `RUN SCRIPT '_environments.etlsql'`.

4. The #Temp Table Workspace

Temporary tables (prefixed with #) are **in-memory engine-side staging areas**. They are the core of every multi-step pipeline.

```
-- Stage data from a remote source
SELECT id, UPPER(name) AS name, email
INTO #stage
FROM pg_source.customers
WHERE updated_at > DATEADD(DAY, -1, GETDATE());

-- Transform it in engine memory
UPDATE #stage SET email = NULL WHERE email NOT LIKE '%@%';

-- Then write to the target
INSERT INTO dest_db.dbo.Customers SELECT * FROM #stage;
```

Why use #temp tables?

1. **Decoupling** — stage data from a slow legacy source before joining it with a modern cloud source
2. **Safety** — validate and clean before executing destructive **MERGE** or **DELETE** operations
3. **Engine-only functions** — apply **REGEX**, **HASHBYTES**, window functions, or **GETDATE()** to data from a source that doesn't natively support them

4.1 Creating and Modifying Temp Tables

```
-- Define structure explicitly
CREATE TABLE #Summary (
    Category VARCHAR(50) NOT NULL,
    Total DECIMAL(18,2),
    LoadedAt DATETIME DEFAULT GETDATE()
);

-- Or create via SELECT INTO
SELECT * INTO #staging FROM source_db.Orders WHERE status = 'Open';

-- Alter structure
ALTER TABLE #staging ADD BatchTag VARCHAR(20);
ALTER TABLE #staging DROP COLUMN LegacyField;

-- Drop when done (auto-dropped at session end anyway)
DROP TABLE IF EXISTS #staging;
```

5. Core SELECT Patterns

5.1 Full Clause Order

```

SELECT [DISTINCT] [TOP n [PERCENT] [WITH TIES]]
    <columns>
[INTO <target>]
FROM <source>
[JOIN ... ON ...]
[WHERE ...]
[GROUP BY ...]
[HAVING ...]
[ORDER BY ... [ASC|DESC]]
[OFFSET n ROWS FETCH NEXT n ROWS ONLY]    -- pagination
[LIMIT n]                                  -- ANSI shorthand
[FOR JSON PATH, ROOT('name')]              -- JSON output

```

5.2 Row Limiting

```

-- T-SQL style
SELECT TOP 10 * FROM #data ORDER BY Amount DESC;

-- ANSI style
SELECT * FROM #data ORDER BY Amount DESC LIMIT 10;

-- Pagination
SELECT * FROM #data ORDER BY id OFFSET 100 ROWS FETCH NEXT 25 ROWS ONLY;

```

5.3 Aggregation with ROLLUP

```

-- Generate subtotals and a grand total in one query
SELECT Region, Product, SUM(Amount) AS Total
FROM #sales
GROUP BY ROLLUP(Region, Product)
ORDER BY Region, Product;
-- NULL in Region = grand total row; NULL in Product = region subtotal row

```

5.4 CTEs (Common Table Expressions)

```

WITH RecentOrders AS (
    SELECT CustomerId, COUNT(*) AS OrderCount, SUM(Amount) AS Total
    FROM #orders
    WHERE OrderDate >= DATEADD(MONTH, -3, GETDATE())
    GROUP BY CustomerId
)
SELECT c.Name, r.OrderCount, r.Total
FROM prod.Customers AS c

```

```
JOIN RecentOrders AS r ON c.Id = r.CustomerId
WHERE r.Total > 5000;
```

6. Control Flow

6.1 IF / ELSE IF / ELSE

```
DECLARE @rowCount INT = (SELECT COUNT(*) FROM #staging);

IF @rowCount = 0
BEGIN
    PRINT 'No data to process. Exiting.';
    RETURN;
END
ELSE IF @rowCount > 1000000
BEGIN
    PRINT 'Large batch detected – enabling chunked processing.';
    -- ... chunked logic
END
ELSE
BEGIN
    INSERT INTO dest_db.Transactions SELECT * FROM #staging;
END
```

6.2 WHILE / FOR / FOREACH Loops

```
-- WHILE: condition-based
DECLARE @retry INT = 0;
WHILE @retry < 3
BEGIN
    -- try operation
    SET @retry = @retry + 1;
    IF @retry = 3 BREAK;
    WAITFOR DELAY '00:00:05';
END

-- FOR: numeric range
FOR @i = 1 TO 12
BEGIN
    PRINT 'Processing month: ' + @i;
END

-- FOREACH: iterate a LIST
DECLARE @regions LIST = ('North', 'South', 'East', 'West');
FOREACH @r IN @regions
BEGIN
    SELECT * INTO #batch FROM prod.Sales WHERE Region = @r;
```

```
INSERT INTO dw.RegionSales SELECT @r AS Region, * FROM #batch;
END
```

6.3 Timing & Polling

```
-- Fixed pause
WAITFOR DELAY '00:00:30';      -- 30 seconds
WAITFOR DELAY '00:00:00.500'; -- 500 milliseconds

-- Wait until a specific time today (or tomorrow if already past)
WAITFOR TIME '02:00:00';

-- Polling form – waits until the expression becomes truthy (polls every 200ms)
WAITFOR (SELECT COUNT(*) FROM control_db.JobStatus WHERE Status = 'Ready');
PRINT 'Condition met – proceeding.';

-- WAIT UNTIL is a more readable alias for the same polling form
WAIT UNTIL (SELECT COUNT(*) FROM control_db.JobStatus WHERE Status = 'Ready') > 0;

-- Equivalent WHILE form – use this when you need control over the poll interval
-- or want to perform additional logic between checks:
DECLARE @ready INT = 0;
WHILE @ready = 0
BEGIN
    SET @ready = (SELECT COUNT(*) FROM control_db.JobStatus WHERE Status =
'Ready');
    IF @ready = 0 WAITFOR DELAY '00:01:00';    -- check every minute
END
PRINT 'Job is ready – proceeding.';
```

7. Error Handling & Transactions

7.1 TRY...CATCH with Rollback

```
BEGIN TRY
    BEGIN TRANSACTION;

    -- Destructive operations inside a transaction
    MERGE INTO prod.Customers AS T
    USING #staging AS S ON T.id = S.id
    WHEN MATCHED THEN UPDATE SET T.name = S.name
    WHEN NOT MATCHED THEN INSERT (id, name) VALUES (S.id, S.name);

    DELETE FROM prod.StagingQueue WHERE Processed = 1;

    COMMIT;
    PRINT 'Load complete.';
END TRY
```



```
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK;
  PRINT 'ERROR: ' + ERROR_MESSAGE();
  THROW; -- re-escalate so the scheduler or parent script sees the failure
END CATCH;
```

7.2 SET WHAT_IF — Safe Dry Runs

Always validate a destructive operation before running it for real:

```
-- Phase 1: Validate – no data is actually changed
SET WHAT_IF ON;
DELETE FROM prod.logs WHERE log_date < '2024-01-01';
MERGE INTO prod.Customers AS T USING #staging AS S ON T.id = S.id
  WHEN MATCHED THEN UPDATE SET T.status = 'Archived';
SET WHAT_IF OFF;

-- Review the console output, then run for real:
DELETE FROM prod.logs WHERE log_date < '2024-01-01';
```

7.3 ASSERT and EXPECT SCHEMA — Proactive Validation

Use these before destructive operations to fail fast with a clear message rather than silently loading bad data.

```
-- ASSERT: halt if a logical condition is false
ASSERT (SELECT COUNT(*) FROM #staging) > 0,
  'Staging table is empty – source feed may have failed';

ASSERT @total_amount >= 0,
  'Negative balance detected – aborting before MERGE';

-- EXPECT SCHEMA: halt (or warn) if the source schema has drifted
EXPECT SCHEMA #staging (
  CustomerId INT,
  Name       VARCHAR,
  Email      VARCHAR,
  Amount     DECIMAL(18,2)
);

-- Warn instead of abort (script continues, but logs a yellow warning)
EXPECT SCHEMA #staging (
  CustomerId INT,
  Name       VARCHAR
) ON DRIFT WARN;
```

[!TIP] Place **ASSERT** and **EXPECT SCHEMA** checks immediately after a source extract and before any **MERGE** or **DELETE**. This pattern catches schema changes from upstream teams before they corrupt your target.

8. Data Movement Patterns

8.1 Extract → Transform → Load (ETL)

The standard pattern — always stage before loading:

```
-- Extract
SELECT id, name, email INTO #raw FROM source_db.Users WHERE active = 1;

-- Transform
UPDATE #raw SET email = LOWER(TRIM(email));
UPDATE #raw SET email = NULL WHERE email NOT LIKE '%@%.%';

-- Load
MERGE INTO dest_db.Users AS T USING #raw AS S ON T.id = S.id
WHEN MATCHED THEN UPDATE SET T.name = S.name, T.email = S.email
WHEN NOT MATCHED THEN INSERT (id, name, email) VALUES (S.id, S.name, S.email);
```

8.2 BULK INSERT — High-Speed File Loading

For large files (1M+ rows), use **BULK INSERT** for O(1) memory usage — it streams without loading the whole file:

```
BULK INSERT dest_db.DailyLogs
FROM 'C:\Incoming\logs_20260412.csv'
WITH (
    FORMAT          = 'CSV',
    FIRSTROW        = 2,          -- skip header row
    BATCHSIZE       = 10000,      -- commit every 10k rows
    MAXERRORS       = 5,          -- allow up to 5 bad rows
    FIELDTERMINATOR = ',',
    STRICT_SCHEMA   = ON
);
```

8.3 EXECUTE — Remote Pushdown

Use **EXECUTE** when you need to run native SQL that isn't representable in ETL-SQL syntax, or to push work to the remote engine:

```
-- The block inside is sent verbatim to the remote (T-SQL dialect here)
EXECUTE prod_db INTO #remote_result
BEGIN
    WITH TopCustomers AS (
        SELECT TOP 100 Id, Name, SUM(Amount) AS Total
        FROM dbo.Orders GROUP BY Id, Name ORDER BY Total DESC
    )
```

```

    SELECT * FROM TopCustomers WHERE Total > 10000;
END;

-- Now work with the result in engine context
SELECT * FROM #remote_result ORDER BY Total DESC;

```

8.4 PARALLEL — Concurrent Execution

Load independent tables simultaneously to reduce wall-clock time:

```

PARALLEL
BEGIN
    BEGIN
        SELECT * INTO #DimDate FROM src.DateDim;
        INSERT INTO dw.DimDate SELECT * FROM #DimDate;
    END
    BEGIN
        SELECT * INTO #DimProduct FROM src.ProductDim;
        INSERT INTO dw.DimProduct SELECT * FROM #DimProduct;
    END
END;
PRINT 'Dimensions loaded in parallel.';

-- Add a concurrency limit to avoid overwhelming a source system
-- At most 3 branches run simultaneously; the rest queue
PARALLEL(3)
BEGIN
    RUN SCRIPT 'load_north.etlsql';
    RUN SCRIPT 'load_south.etlsql';
    RUN SCRIPT 'load_east.etlsql';
    RUN SCRIPT 'load_west.etlsql';
END;

```

8.5 Dynamic SQL with EXEC

Build and execute SQL statements constructed at runtime. The engine supports five EXEC forms:

```

-- Form 1: Dynamic expression – EXEC(@string_expr)
-- Parses and runs the string in ENGINE context. Full access to #temp tables and
@variables.
DECLARE @tbl = '#orders';
DECLARE @sql = 'SELECT COUNT(*) AS Total FROM ' + @tbl + ' WHERE Status =
''Open''';
EXEC(@sql);
-- Optional: capture result
EXEC(@sql) INTO #result;

-- Form 2: Dynamic expression against a remote connection – EXEC(@expr) AT conn
DECLARE @archiveTable = 'Archive_' + CAST(YEAR(GETDATE()) AS STRING);

```

```

DECLARE @sql = 'SELECT TOP 100 * FROM dbo.' + @archiveTable + ' ORDER BY Id
DESC;';
EXEC(@sql) AT prod_db INTO #archive;

-- Form 3: Parameterized remote query – EXEC(@expr) AT conn WITH(param1, param2,
...)
-- Parameters bind to @p0, @p1, ... in the SQL string (safe – avoids SQL
injection)
EXEC('SELECT * FROM dbo.Orders WHERE Status = @p0 AND Year = @p1') AT prod_db
    WITH('Active', YEAR(GETDATE()))
    INTO #active_orders;

-- Form 4: Remote block pushdown – EXEC conn BEGIN ... END
-- Passes the block verbatim to the remote engine (native dialect). Same as
EXECUTE ... BEGIN...END.
EXEC prod_db INTO #result
BEGIN
    WITH ranked AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerId ORDER BY
OrderDate DESC) AS rn FROM dbo.Orders)
    SELECT * FROM ranked WHERE rn = 1;
END;

-- Form 5: Stored procedure call – EXEC proc_name [params]
EXEC LoadRegion 'North', '2026-01-01';
EXEC dbo.sp_archive @Year = 2025, @DryRun = 1;

```

[!IMPORTANT] `EXEC(@sql)` without `AT conn` runs in **engine context** — it can read `#temp` tables and `@variables` from the current session. `EXEC(@sql) AT conn` sends the SQL string to the remote engine and cannot reference engine-side objects. `WHAT_IF` mode is respected: remote EXEC statements log what would run without executing.

[!TIP] **Choosing the right form:** Use Form 1 for dynamic ETL logic that needs engine functions. Use Form 3 (parameterized) any time user input or variable values appear in the query string — it eliminates SQL injection risk. Use Form 4 when you need stored procs, CTEs, or native syntax the ETL-SQL parser doesn't handle directly.

9. File Operations

Every file path must be **absolute**. Always check existence before operating.

```

DECLARE @src = 'C:\Incoming\data.csv';
DECLARE @dest = 'C:\Archive\data.csv';

IF NOT FILE_EXISTS(@src)
BEGIN
    PRINT 'Source file not found: ' + @src;
    RETURN;
END

-- Copy, compress, encrypt, then transmit

```

```

COPY FILE      @src TO @dest WITH(OVERWRITE=ON);
COMPRESS FILE  @dest TO @dest + '.gz' WITH(OVERWRITE=ON);
ENCRYPT FILE    @dest + '.gz' TO @dest + '.enc' PASSWORD('vaultkey')
WITH(OVERWRITE=ON);

-- Upload to SFTP and clean up
SEND FILE @dest + '.enc' TO '/outbox/' AT sftp_conn;
DELETE FILE @dest + '.gz';
DELETE FILE @dest + '.enc';

```

File Listing

```

-- List local files
SELECT Name, Size, LastModified
FROM FILE_LIST('C:\Incoming', '*.csv')
ORDER BY LastModified DESC;

-- List remote SFTP files
SELECT Name, Size, LastModified
INTO #remote_manifest
FROM REMOTE_FILE_LIST(sftp_conn, '/uploads/')
WHERE LastModified >= DATEADD(HOUR, -24, GETDATE());

```

[!TIP] For the full file operation reference — [COMPRESS FILE](#), [ENCRYPT FILE](#), [MOVE FILE](#), directory operations, and SFTP/FTP patterns — see [Specialized_Operations.md](#).

10. Modular Scripts & Jobs

10.1 Breaking Scripts into Modules

```

-- orchestrator.etlsql
DECLARE @env STRING INPUT = 'DEV';

RUN SCRIPT 'extract.etlsql' WITH (@env = @env);
RUN SCRIPT 'transform.etlsql' WITH (@env = @env);
RUN SCRIPT 'load.etlsql' WITH (@env = @env);

```

10.2 Scheduling with CREATE JOB

```

-- Run every night at 2 AM
CREATE JOB NightlyLoad ON SCHEDULE EVERY 1 DAY AT '02:00' AS
BEGIN
    RUN SCRIPT 'C:\Scripts\nightly_load.etlsql';
END;

```

```
-- Monitor
SHOW JOBS;
SHOW JOB HISTORY NightlyLoad;
```

10.3 Procedures for Reusable Logic

```
CREATE PROCEDURE LoadRegion @Region STRING, @StartDate DATE
AS
BEGIN
    SELECT * INTO #batch
    FROM prod.Sales
    WHERE Region = @Region AND SaleDate >= @StartDate;

    INSERT INTO dw.RegionSales SELECT @Region, * FROM #batch;
    PRINT 'Loaded ' + @Region + ' from ' + CAST(@StartDate AS STRING);
END;

EXEC LoadRegion 'North', '2026-01-01';
EXEC LoadRegion 'South', '2026-01-01';
```

[!TIP] For the full scheduling reference — [SHOW JOBS](#), [DROP JOB](#), [KILL JOB](#), CI/CD integration, and Windows Service deployment — see the [Orchestrator's Guide](#).

11. Metadata, Lineage & Tags

ETL-SQL automatically tracks where every piece of data came from. Tag columns and tables using inline `/* ... */` comments:

```
SELECT
    UserId    /* @d: Internal surrogate key; @PII: false; */,
    Email     /* @d: User email address; @PII: true; @owner: SecurityTeam; */,
    Region    /* @d: Sales territory code; */
INTO #TaggedUsers
FROM prod.Users /* @sensitivity: medium; */;

-- View the lineage tree
LINEAGE(#TaggedUsers);

-- Export a Mermaid diagram + audit table
LINEAGE(#TaggedUsers) TO 'C:\Reports\user_lineage.md';

-- Query lineage programmatically
SELECT Operation, SourceTables, TargetColumn
FROM LINEAGE(#TaggedUsers)
WHERE TargetColumn = 'Email';
```

11.1 Script Metadata Headers

The engine automatically reads a structured comment at the top of any `.etlsql` file and records the metadata in lineage logs:

```
/*
  @author:      Chuck
  @version:     2.1.0
  @description: Nightly customer sync from Postgres → SQL Server DW
*/

DECLARE @BatchDate DATE = GETDATE();
...
```

Any `@key: value` pair is captured. `@author` defaults to the current system user if omitted. This metadata appears in `SHOW LINEAGE` output and the Orchestrator job history.

12. Debugging & Diagnostics

12.1 LINT — Before You Run

```
-- Analyze a script for errors and best-practice violations before executing
LINT 'C:\Scripts\nightly_load.etlsql';
```

12.2 EXPLAIN — Query Plans

```
EXPLAIN
SELECT o.OrderId, c.Name
FROM prod.Orders AS o
JOIN prod.Customers AS c ON o.CustomerId = c.Id
WHERE o.Status = 'Open';
```

12.3 PROFILING — Performance Measurement

```
SET PROFILING ON;
RUN SCRIPT 'C:\Scripts\heavy_transform.etlsql';
SET PROFILING OFF;

-- View top 10 slowest operations
SHOW PROFILE INTO #perf;
SELECT * FROM #perf ORDER BY DurationMs DESC LIMIT 10;
```

[!TIP] @@LAST_EXEC_MS, @@TOTAL_SPILLED_BYTES, and @@PEAK_MEMORY_MB are system variables you can query at any time — no profiling mode required. See [Administrators_Guide.md §5.4](#) for the full system variable reference.

12.4 MOCKDB — Safe Development

Use the built-in mock database for development without touching production:

```
CREATE CONNECTION m ON MOCKDB();

-- Pre-seeded tables: Users, Products, Orders, Employee, departments
SELECT * FROM m.Users;
SELECT u.UserName, o.TotalAmount FROM m.Users AS u JOIN m.Orders AS o ON u.UserID
= o.OrderID;
```

13. Zero-Trust Security

ETL-SQL treats every script as untrusted by default regardless of who wrote it or where it came from. The sandbox rules below are always enforced — they cannot be disabled, only selectively relaxed inside an administrator-approved [Safe Zone](#). See [SECURITY.md](#) for the complete policy.

Key rules enforced by the engine:

Rule	Details
No system directory access	Windows , System32 , .ssh , .git , .aws are permanently blocked
No root drive access	Operations targeting C:\ or / directly are blocked
No script self-modification	Writing .etlsql , .sql , .py , .js , .sh files is blocked
No dangerous file types	.dll , .exe , .bat , .cmd , .sh , .msi cannot be read or written
Operation cap	Max 100 filesystem operations per script; max 5 recursive directory levels
Credential masking	Never PRINT passwords, tokens, or ENC: values

Use [SET WHAT_IF ON](#) before any destructive operation. See [\[SECURITY.md\]\(file:///c:/Users/chuck/scratch/ETL-SQL/SECURITY.md\)](#) for the complete security policy.

Next Steps

Topic	Document
Full language syntax — every keyword	[Grammar.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Reference/Grammar.md)
Connector options and authentication	[Data_Connectors.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Reference/Data_Connectors.md)

Topic	Document
All built-in functions	[Standard_Library.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Reference/Standard_Library.md)
File ops, email, lineage, Docker, jobs	[Specialized_Operations.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Reference/Specialized_Operations.md)
18 production-ready recipes	[Cookbook.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Cookbook.md)
55+ sample scripts inventory	[Sample_Guide.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Sample_Guide.md)
Reporting & dashboards	[Report_SQL_Guide.md](file:///c:/Users/chuck/scratch/ETL-SQL/Docs/Report_SQL_Guide.md)
Security policy	[SECURITY.md](file:///c:/Users/chuck/scratch/ETL-SQL/SECURITY.md)